Compte Rendu #5 : Post-traitement et Connectivité

Mathys CHAMPENOIS, Cesar CHARBEY, Mickael CZOLACZ - Super Pixels

Dépôt GitHub : `https://github.com/Mathys3114/ProjetCCMTI`

1 Bilan de la semaine :

Lors de nos précédent ajouts, nous avions identifié un défaut à l'algorithme SLIC de base : la présence de pixels isolés et de superpixels éclatés en plusieurs régions non contiguës. Notre objectif principal était d'implémenter la phase de post-traitement pour régler ce problème.

Cet objectif a été pleinement atteint. L'algorithme est désormais capable de forcer la connectivité spatiale des superpixels, garantissant une segmentation parfaitement propre, sans aucun ni fragment.

2 L'algorithme de Connectivité étape par étape

Conformément à la section 4 du papier de référence, nous avons implémenté la correction de la connectivité en quatre algorithmes distincts :

1. **Étiquetage des composantes connexes (Algo 6)** : L'algorithme parcourt la carte des labels générée par le K-Means local pour identifier les blocs physiques continus via un parcours en largeur. Les résultats sont stockés dans une structure `CCResult`. Nous construisons également un graphe d'adjacence utilisant `std::map«int, std::set«int»»` pour répertorier les voisins de chaque région.
2. **Isolement des orphelins (Algo 7)** : Pour chaque superpixel d'origine, nous conservons uniquement la composante connexe possédant la plus grande aire. Tous les autres pixels appartenant aux composantes mineures se voient attribuer un label invalide (-1) et deviennent des orphelins.
3. **Calcul des distances (Algo 8)** : Afin d'éviter qu'un pixel orphelin n'adopte la couleur d'un autre orphelin, nous calculons la distance de Manhattan entre chaque orphelin et le pixel validé le plus proche via des dilatations successives.
4. **Adoption (Algo 9)** : Les orphelins sont traités dans l'ordre croissant de leur distance. Chaque orphelin est réaffecté (adopté) au superpixel voisin valide dont la couleur moyenne est la plus proche de sa propre couleur.

3 Exécutables et Visualisation

Pour valider cette étape, nous avons mis à jour notre pipeline avec un exécutable de test :

— `slicCC` : Ce programme permet d'exécuter l'algorithme SLIC puis de visualiser l'état de la carte des labels juste après l'étiquetage des composantes connexes (par colorisation aléatoire de chaque fragment). Il applique ensuite les étapes de nettoyage pour générer l'image finale propre.

4 Visualisation des résultats

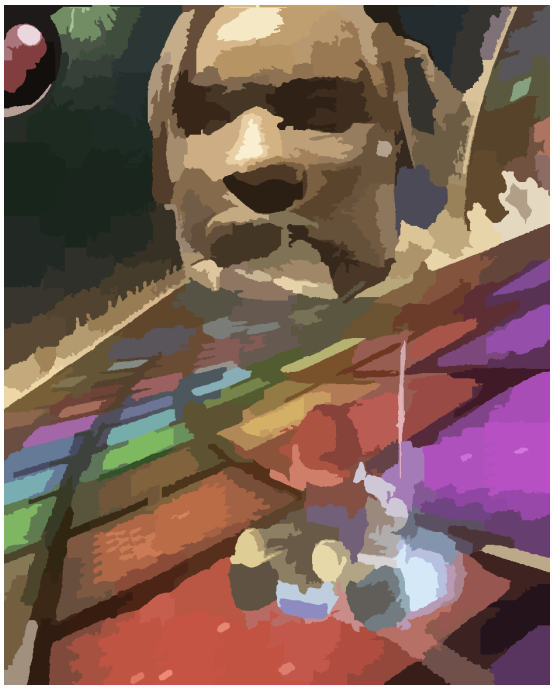
Afin de vérifier la robustesse de notre algorithme d'application de la connectivité, nous avons généré deux jeux de tests.

4.1 Test de contrainte (Génération massive d'orphelins)

Dans ce premier test ($K = 300, m = 5, g = 0$), le paramètre de compacité m est volontairement très faible. La similarité de couleur domine largement la distance spatiale lors du K-Means, ce qui provoque l'éclatement des superpixels en de multiples fragments isolés. Notre algorithme parvient à réaffecter tous ces fragments pour reconstruire des blocs parfaitement continus.

(a) SLIC Brut (Fragments isolés)

(b) Composantes Connexes (Avant nettoyage)



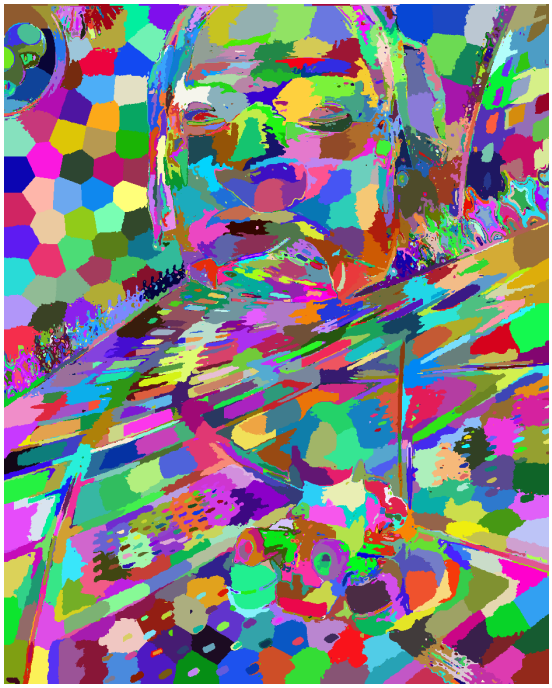
(c) Résultat Propre

FIGURE 1 – Résultats avec les paramètres : $K = 300$, $m = 5$

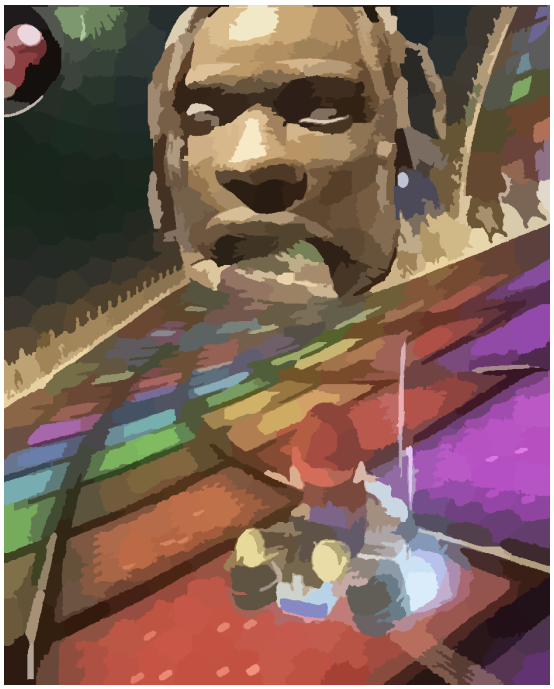
4.2 Conditions normales (Paramètres recommandés)

Dans ce second test ($K = 600, m = 30, g = 0$), les paramètres sont équilibrés et proches des recommandations de l'article. Les fragments générés par le K-Means sont beaucoup moins nombreux et se concentrent logiquement sur les contours des objets. L'algorithme de connectivité absorbe proprement ces petits orphelins sans dénaturer la segmentation globale.

(a) SLIC Brut (Contours instables)



(b) Composantes Connexes (Avant nettoyage)



(c) Résultat Propre (Après application de la connectivité)

FIGURE 2 – Résultats avec des paramètres normaux : $K = 600$, $m = 30$